

Local AI Memory Assistant

Fases, términos e importancia de cada implementación

Documento técnico orientado a reutilización: qué hace cada pieza, por qué existe, qué mide, cómo se integra y cuándo aporta valor en otros sistemas de IA, búsqueda, memoria y recomendación.

Alcance. Este documento describe el proyecto por fases funcionales: evaluación de retrieval, búsqueda híbrida, ranking LTR, reranking, scoring de calidad, deduplicación, diversidad, feedback, aprendizaje online/offline, persistencia del modelo, contexto, knowledge graph, lifecycle, jobs, observabilidad y pruebas.

Principio central: el valor del proyecto no está en una tecnología aislada, sino en el pipeline completo que convierte memoria persistida en candidatos, los ordena con múltiples señales, aprende de feedback y mide si la calidad realmente mejora.

1. Visión global del pipeline

El camino canonical de retrieval del proyecto puede resumirse como: Query → análisis/estrategia → Hybrid Retrieval → fusión → LTR Ranking → Reranking → Quality Scoring → Duplicate Detection → Diversity Filtering → Feedback Reranking → métricas → feedback → entrenamiento → persistencia del modelo.

La arquitectura observada combina memoria vectorial/keyword/grafó, Weighted Reciprocal Rank Fusion, LTR con features explícitas, reranking por embeddings, scoring de calidad, filtros de duplicados y diversidad, feedback persistido, entrenamiento offline y actualización online. Además existen capas de contexto, knowledge graph, inferencia, lifecycle y jobs.

Fase	Entrada	Salida principal	Objetivo
1. Query analysis	Consulta	Perfil/estrategia	Decidir cómo recuperar
2. Hybrid Retrieval	Consulta + estrategia	Candidatos	Maximizar cobertura
3. Fusion / RRF	Rankings parciales	Ranking fusionado	Combinar fuentes
4. LTR Ranking	Candidatos + features	Score aprendido	Ordenar relevancia
5. Reranking	Top candidatos	Orden refinado	Mejorar semántica final
6. Quality Scoring	Candidato	Score de calidad	Evitar resultados débiles
7. Duplicate Detection	Resultados	Resultados únicos	Eliminar redundancia
8. Diversity Filtering	Resultados	Lista diversa	Evitar repetición temática
9. Feedback Reranking	Feedback histórico	Ajuste ranking	Personalizar/aprender
10. Evaluation	Ranking + ground truth	P@K/R@K/MRR/NDCG	Medir calidad

2. Fase de evaluación: métricas de ranking

Estas métricas son críticas porque separan una impresión subjetiva de una mejora medible. Sin ellas, cambiar pesos, añadir LTR o introducir contexto puede parecer mejor aunque degrade el ranking real.

Precision@K

Qué es: Proporción de resultados relevantes dentro de los primeros K resultados. Fórmula: $\text{Precision@K} = \frac{\text{relevantes_en_top_K}}{K}$.

Importancia en este proyecto: Mide la limpieza de la parte visible del ranking. Es especialmente útil cuando el usuario solo inspecciona pocos resultados.

Valor en otros proyectos: En buscadores, RAG, recomendadores y memoria contextual permite fijar una puerta de calidad objetiva antes de desplegar cambios.

Recall@K

Qué es: Proporción de todos los resultados relevantes disponibles que aparecen dentro de los primeros K. Fórmula: $\text{Recall@K} = \frac{\text{relevantes_en_top_K}}{\text{relevantes_totales}}$.

Importancia en este proyecto: Mide cobertura. Un sistema puede tener alta precisión pero perder memorias importantes; Recall@K revela ese problema.

Valor en otros proyectos: Es esencial cuando omitir información relevante es costoso: soporte, investigación, búsqueda documental o recuperación de contexto.

MRR - Mean Reciprocal Rank

Qué es: Promedio del inverso de la posición del primer resultado relevante: $1/\text{rank}$. Si el primer relevante está en posición 1, aporta 1; en posición 2, 0.5.

Importancia en este proyecto: Evalúa qué tan rápido aparece la primera respuesta útil. Penaliza con fuerza cuando el primer relevante baja posiciones.

Valor en otros proyectos: Ideal para asistentes, QA, búsqueda de entidades y flujos donde el usuario espera que el primer resultado sea correcto.

NDCG@K

Qué es: Normalized Discounted Cumulative Gain. Valora relevancia graduada y descuenta resultados relevantes que aparecen tarde; se normaliza frente al ranking ideal.

Importancia en este proyecto: Es más expresiva que Precision/Recall cuando existen diferentes grados de relevancia y el orden exacto importa.

Valor en otros proyectos: Muy útil en LTR, recomendación, búsqueda empresarial y cualquier sistema donde varios resultados son relevantes pero no igualmente buenos.

Interpretación conjunta: Precision@K mide pureza, Recall@K cobertura, MRR la posición del primer acierto y NDCG@K la calidad global del orden. Ninguna debería sustituir a las demás.

3. Fase de Hybrid Retrieval

Vector Retrieval / Semantic Retrieval

Qué es: Recuperación mediante embeddings y similitud vectorial.

Importancia en este proyecto: Encuentra memorias semánticamente relacionadas aunque no compartan las mismas palabras.

Valor en otros proyectos: Base de RAG, buscadores semánticos, asistentes de soporte y recuperación de conocimiento.

Keyword / BM25 Retrieval

Qué es: Recuperación léxica que favorece coincidencias exactas o estadísticamente importantes de términos.

Importancia en este proyecto: Complementa a embeddings cuando nombres, identificadores, siglas o frases exactas deben dominar.

Valor en otros proyectos: Muy valiosa en documentación técnica, logs, código, catálogos y consultas con términos exactos.

Graph Retrieval

Qué es: Recuperación de nodos y relaciones desde un grafo de conocimiento.

Importancia en este proyecto: Añade evidencia relacional: no solo qué textos se parecen, sino qué entidades están conectadas.

Valor en otros proyectos: Útil en sistemas de conocimiento, dependencias, entidades, recomendaciones relacionales y reasoning multi-hop.

Graph Evidence Retrieval

Qué es: Señal adicional que cuantifica evidencia derivada de relaciones o inferencias del grafo.

Importancia en este proyecto: Permite que candidatos apoyados por estructura relacional obtengan prioridad aunque su similitud textual sea menor.

Valor en otros proyectos: Aporta valor cuando las relaciones entre conceptos importan tanto como el contenido textual.

Weighted Reciprocal Rank Fusion (Weighted RRF)

Qué es: Fusión de rankings parciales mediante la posición de cada candidato y pesos por fuente.

Importancia en este proyecto: Evita depender de una única escala de score y hace robusta la combinación vector + keyword + graph + graph evidence.

Valor en otros proyectos: Es una técnica reutilizable cuando se combinan motores heterogéneos con scores no comparables.

Por qué es importante el enfoque híbrido: los embeddings maximizan semántica; BM25 protege coincidencia exacta; el grafo aporta relaciones; RRF integra todo sin exigir que cada subsistema tenga la misma escala de puntuación.

4. Fase de estrategia adaptativa de retrieval

Query Profile

Qué es: Representación de características de la consulta: tokenCount, specificity, complexity, semanticIntent, keywordIntent, relationalIntent, temporalIntent, exact terms, entities y comparisonIntent.

Importancia en este proyecto: Permite decidir qué fuente merece más peso según la consulta.

Valor en otros proyectos: Evita una estrategia fija para todas las búsquedas; se puede reutilizar en motores multi-modalidad o multi-índice.

Retrieval Strategy

Qué es: Configuración derivada del perfil: mode, vectorWeight, keywordWeight, graphWeight, graphEvidenceWeight, topK, expansión de query, rerank y temporalBoost.

Importancia en este proyecto: Controla el candidate budget y el equilibrio precisión/cobertura antes de aplicar ranking avanzado.

Valor en otros proyectos: Es útil en cualquier sistema donde consultas cortas, relacionales, temporales o exactas necesiten tratamientos diferentes.

Ejemplo del proyecto: una consulta relacional como “¿qué relación existe entre Angular y TypeScript?” puede activar un modo hybrid_graph con peso explícito para graph y graph evidence, mientras una frase exacta puede inclinarse hacia keyword retrieval.

5. Fase de Learning to Rank (LTR)

FeatureExtractor

Qué es: Convierte cada par query-candidato en un vector de señales numéricas.

Importancia en este proyecto: Hace explícitas las razones que influyen en el ranking y desacopla la extracción de señales del modelo.

Valor en otros proyectos: Permite evolucionar el modelo sin reescribir todo el retrieval y facilita auditoría/experimentos.

Feature	Significado	Por qué importa
semantic	Similitud semántica query-memoria	Relevancia conceptual
bm25	Coincidencia léxica	Exactitud de términos
importance	Importancia acumulada de la memoria	Prioridad del conocimiento
confidence	Confianza en la memoria	Evita priorizar datos dudosos
freshness	Recencia	Favorece información actual cuando corresponde
graphEvidence	Soporte relacional del grafo	Añade evidencia estructural
accessCount	Uso histórico	Captura utilidad recurrente
diversity	Aporte de diversidad	Reduce redundancia de lista
duplicatePenalty	Penalización por duplicado	Desciende candidatos repetidos
feedbackScore	Señal agregada de feedback	Incorpora comportamiento real
retrievalFrequency	Frecuencia de recuperación	Señal de utilidad/uso
ageScore	Transformación de antigüedad	Permite controlar decay temporal

LinearModel / LTRRanker

Qué es: Modelo que combina features con pesos aprendidos para producir un score final de ranking.

Importancia en este proyecto: Reemplaza heurísticas completamente manuales por un orden que puede aprender a partir de ejemplos y feedback.

Valor en otros proyectos: Es un punto de partida interpretable para ranking personalizado antes de evolucionar a modelos más complejos.

PersistentLTRModelProvider + ModelRepository

Qué es: Capa que carga el modelo persistido y lo expone al ranker.

Importancia en este proyecto: Permite que el sistema use el último modelo entrenado tras reinicios, en lugar de volver a pesos por defecto.

Valor en otros proyectos: Patrón reutilizable en cualquier servicio ML que necesite desacoplar inferencia de almacenamiento del modelo.

Ejemplo de vector de entrenamiento

query	angular federation
memoryId	memory-001
semantic	0.92
bm25	0.81
importance	0.70
confidence	0.90
freshness	0.95
graphEvidence	0.61
accessCount	0.42
diversity	0.84
duplicatePenalty	0.05
label	1

El label representa la señal objetivo usada para aprender. El proyecto también contiene un reward model con señales como click, accept, reject e ignore; por eso la semántica del feedback debe distinguir exposición de acción explícita.

6. Fase de reranking

Embedding Reranker

Qué es: Reordena un conjunto reducido de candidatos usando una señal semántica más costosa o precisa.

Importancia en este proyecto: Es eficiente porque se aplica después del retrieval inicial, no sobre todo el corpus.

Valor en otros proyectos: Patrón estándar en sistemas de búsqueda de dos etapas: retrieve barato, rerank preciso.

Semantic Reranker

Qué es: Refina el orden según relevancia semántica después de fusionar fuentes.

Importancia en este proyecto: Compensa casos donde RRF conserva candidatos por cobertura pero el orden final aún no es óptimo.

Valor en otros proyectos: Reutilizable en RAG, búsqueda, recomendación y deduplicación semántica.

FeedbackDrivenReranker

Qué es: Ajusta ranking usando señales históricas guardadas en FeedbackRepository.

Importancia en este proyecto: Permite que el sistema aprenda preferencias reales sin depender solo del modelo base.

Valor en otros proyectos: Útil para personalización, búsqueda corporativa y productos con señales de interacción.

7. Fase de Quality Scoring

QualityScoringService

Qué es: Calcula una valoración adicional de la calidad del candidato, separada de relevancia pura.

Importancia en este proyecto: Un resultado puede ser relevante pero poco fiable, desactualizado o de baja calidad; esta capa permite filtrarlo o penalizarlo.

Valor en otros proyectos: Se reutiliza en RAG para evitar que contexto débil llegue al LLM, y en recomendadores para imponer estándares mínimos.

En el proyecto se registran métricas de calidad como score medio, relevancia media, confianza media y duplicados eliminados. Esto permite observar no solo latencia, sino también la calidad del conjunto final.

8. Duplicate Detection y Diversity Filtering

DuplicateDetector

Qué es: Detecta resultados idénticos o muy similares mediante TextSimilarityService u otra señal.

Importancia en este proyecto: Evita desperdiciar posiciones del top-K con memorias equivalentes.

Valor en otros proyectos: Fundamental en memoria acumulativa, RAG y bases donde el mismo hecho puede almacenarse varias veces.

Duplicate Penalty

Qué es: Feature o penalización aplicada a candidatos que muestran alta similitud con otros resultados.

Importancia en este proyecto: Permite que LTR aprenda a bajar duplicados incluso antes de un filtro duro.

Valor en otros proyectos: Es reutilizable cuando se desea un equilibrio entre mantener variantes útiles y eliminar clones.

DiversityService / Diversity Filtering

Qué es: Selecciona un conjunto final con menor redundancia temática o semántica.

Importancia en este proyecto: Aumenta la cobertura informativa dentro de un top-K limitado.

Valor en otros proyectos: Muy valioso en recomendación, resultados exploratorios, RAG y resumen multi-fuente.

Relación entre ambas: deduplicación elimina casi-equivalentes; diversidad evita que resultados distintos hablen todos de lo mismo.

9. Fase de feedback y aprendizaje

FeedbackCollector

Qué es: Captura interacción asociada a una query, memoria, tipo de feedback, features y señal.

Importancia en este proyecto: Conecta el comportamiento del usuario con el entrenamiento del ranking.

Valor en otros proyectos: Es el puente necesario para pasar de un ranking estático a uno adaptativo.

FeedbackRepository

Qué es: Persistencia de eventos de feedback.

Importancia en este proyecto: Permite entrenar offline, auditar señales y reconstruir datasets.

Valor en otros proyectos: Patrón clave en sistemas de recomendación, búsqueda y personalización.

FeedbackService

Qué es: Capa de negocio para registrar y consultar feedback.

Importancia en este proyecto: Evita acoplar endpoints o pipeline directamente al formato de almacenamiento.

Valor en otros proyectos: Facilita cambiar almacenamiento, validación o política sin afectar consumidores.

FeedbackLearningService

Qué es: Orquesta feedback y entrenamiento online.

Importancia en este proyecto: Convierte una señal de usuario en una actualización controlada del modelo.

Valor en otros proyectos: Reutilizable cuando el producto debe aprender continuamente sin un batch manual.

Reward Model

Qué es: Mapea acciones a recompensa numérica; en pruebas del proyecto aparecen ejemplos como click=0.30, accept=1.00, reject=-1.00 e ignore=-0.20.

Importancia en este proyecto: Da significado cuantitativo a acciones heterogéneas.

Valor en otros proyectos: Útil para online learning, bandits, recomendación y priorización basada en interacción.

Riesgo semántico importante: una impresión o resultado mostrado no debe tratarse automáticamente como click/aceptación. Si se hace, el modelo aprende una señal positiva falsa y puede reforzar su propio sesgo.

10. Entrenamiento offline y online

TrainingService - Offline Training

Qué es: Entrena o recalcula el modelo a partir del feedback acumulado.

Importancia en este proyecto: Permite aprender con un dataset más amplio, reproducir experimentos y versionar cambios.

Valor en otros proyectos: Adecuado para batch retraining, validación antes de producción y recuperación ante degradación.

OnlineTrainer

Qué es: Aplica actualizaciones incrementales al modelo conforme llegan señales.

Importancia en este proyecto: Reduce el tiempo entre feedback y adaptación del ranking.

Valor en otros proyectos: Útil en sistemas dinámicos, personalización y dominios donde preferencias cambian rápido.

OnlineOptimizer

Qué es: Calcula cómo modificar los pesos a partir del error/recompensa.

Importancia en este proyecto: Encapsula la matemática de actualización y permite cambiar la estrategia de optimización.

Valor en otros proyectos: Patrón reutilizable para SGD, gradients, bandits u optimizadores propios.

LearningRate

Qué es: Controla el tamaño de las actualizaciones online.

Importancia en este proyecto: Evita que una sola interacción cambie el ranking de forma excesiva.

Valor en otros proyectos: Necesario en cualquier aprendizaje incremental para equilibrar adaptación y estabilidad.

Las pruebas del proyecto muestran pesos antes/después de una actualización y deltas por feature. Esto es importante porque hace observable el aprendizaje y permite detectar explosiones, deriva o señales mal escaladas.

11. Persistencia del modelo y restauración

Componente / capacidad	Estado
FeedbackCollector	OK
FeedbackRepository	OK
FeedbackService	OK
FeedbackLearningService	OK
OnlineTrainer	OK
OnlineOptimizer	OK
ModelRepository	OK
Persistencia del modelo	OK
Restauración tras restart	OK
Integración LTRRanker	OK
Eliminación loader antiguo	OK
Tests regresión	OK

La persistencia del modelo es una implementación crítica porque separa entrenamiento de serving. Un modelo entrenado que se pierde al reiniciar no constituye aprendizaje operativo.

A nivel de producción, además de persistir pesos LTR, el proyecto debe garantizar que feedback, knowledge graph, inferencias y otros JSON operativos sobrevivan despliegues y reinicios del contenedor.

12. Tests de regresión de ranking

Regression Ranking Tests

Qué es: Datasets versionados con queries, resultados esperados y umbrales mínimos de métricas.

Importancia en este proyecto: Evitan que una mejora local en una feature degrade consultas previamente correctas.

Valor en otros proyectos: Son indispensables para cambios en embeddings, pesos, LTR, reranking, contexto o índices.

La regla recomendada es comparar baseline vs candidato con Precision@K, Recall@K, MRR y NDCG@K, además de latencia. La aceptación de un cambio debería depender de umbrales explícitos, no solo de que

el pipeline “funcione”.

13. Context-aware retrieval

ContextModel

Qué es: Modelo estructurado derivado de la consulta y/o sesión: entidades, temas, goals, restricciones, temporalidad u otras señales.

Importancia en este proyecto: Permite que la estrategia de retrieval tenga información más rica que el texto literal de la query.

Valor en otros proyectos: Muy valioso en asistentes multi-turn, agentes, CRM, copilots y memoria personal.

ContextBuilder / Context Orchestrator

Qué es: Construye el contexto utilizable por retrieval y generación.

Importancia en este proyecto: Separa extracción de contexto de su consumo, facilitando pruebas y evolución.

Valor en otros proyectos: Patrón reusable en RAG conversacional y sistemas con estado.

Candidate Budget

Qué es: Número de candidatos recuperados antes del ranking final.

Importancia en este proyecto: Queries ambiguas pueden necesitar más cobertura; queries exactas pueden usar un budget menor.

Valor en otros proyectos: Permite controlar costo/latencia y adaptar recall dinámicamente.

En la auditoría del proyecto, la capa context-aware existe pero aparece como integración parcial: el pipeline admite contexto, aunque el camino HTTP debe asegurar que el ContextModel construido realmente llegue al retrieval.

14. Knowledge Graph, conocimiento e inferencia

KnowledgeRepository

Qué es: Persistencia de conocimiento extraído o consolidado.

Importancia en este proyecto: Diferencia memoria textual de conocimiento estructurado.

Valor en otros proyectos: Reutilizable en sistemas que necesitan razonamiento, normalización o trazabilidad de hechos.

KnowledgeSyncService

Qué es: Sincroniza conocimiento persistido con nodos y aristas del grafo.

Importancia en este proyecto: Evita que retrieval de grafo e inferencia trabajen con una representación desactualizada.

Valor en otros proyectos: Es un patrón útil cuando existen dos modelos de almacenamiento que deben permanecer coherentes.

GraphRepository / GraphStorage

Qué es: Acceso y persistencia del grafo.

Importancia en este proyecto: Centraliza identidad, relaciones y almacenamiento.

Valor en otros proyectos: Base de grafos locales, knowledge graphs empresariales y reasoning estructural.

InferenceRepository / Multi-hop inference

Qué es: Persiste o gestiona conocimiento derivado a partir de relaciones.

Importancia en este proyecto: Permite recuperar no solo hechos explícitos, sino conclusiones derivadas.

Valor en otros proyectos: Aplicable a recommendation, troubleshooting, dependencias, investigación y agentes.

Una integración crítica pendiente identificada en el proyecto es garantizar el flujo automático extracción de conocimiento → persistencia → sincronización de grafo → validación → inferencia.

15. Memory lifecycle y consolidation

Memory Lifecycle

Qué es: Políticas que actualizan importancia, accessCount, lastAccess, decay, archivado y otros estados durante la vida de una memoria.

Importancia en este proyecto: Evita que el sistema trate todos los recuerdos como igualmente útiles para siempre.

Valor en otros proyectos: Reutilizable en memoria de asistentes, caches inteligentes, sistemas personales y knowledge bases.

Importance

Qué es: Señal de prioridad de una memoria, que puede aumentar por uso o reforzamiento.

Importancia en este proyecto: Aporta una dimensión de utilidad más allá de similitud con la query.

Valor en otros proyectos: Se usa en ranking, retención y consolidación.

Freshness / Age Score

Qué es: Transformación de edad o recencia de una memoria.

Importancia en este proyecto: Permite preferir información reciente en dominios donde lo nuevo importa sin borrar conocimiento histórico.

Valor en otros proyectos: Aplicable a noticias, operaciones, incidentes, preferencias y datos temporales.

ConsolidationService

Qué es: Fusiona o resume memorias relacionadas para reducir fragmentación.

Importancia en este proyecto: Controla crecimiento, redundancia y deriva del almacenamiento.

Valor en otros proyectos: Muy útil en asistentes de largo plazo y sistemas que acumulan eventos repetidos.

Cleanup Job

Qué es: Proceso que elimina o depura memorias elegibles según una política explícita.

Importancia en este proyecto: Evita crecimiento indefinido, pero debe ser idempotente, trazable y seguro.

Valor en otros proyectos: Reutilizable en sistemas con retención, archivado y costes de almacenamiento.

En la auditoría v1, el cleanup figura como incompleto/no-op; por tanto su diseño debe priorizar dry-run, métricas y reglas estrictas antes de borrar datos.

16. Jobs, scheduler y operación

Scheduler

Qué es: Planifica tareas periódicas como inference, relearning, cleanup o training.

Importancia en este proyecto: Convierte funcionalidades aisladas en comportamiento operativo continuo.

Valor en otros proyectos: Reutilizable en cualquier servicio que necesite mantenimiento, sincronización o aprendizaje periódico.

runJob + JobRepository

Qué es: Wrapper para ejecutar trabajos y registrar estado/historial.

Importancia en este proyecto: Mejora observabilidad, diagnóstico y recuperación.

Valor en otros proyectos: Patrón general para background jobs, ETL, sincronización y ML pipelines.

Training Job

Qué es: Ejecución programada del entrenamiento LTR.

Importancia en este proyecto: Permite incorporar feedback acumulado sin intervención manual.

Valor en otros proyectos: Reutilizable para modelos recalibrados periódicamente.

La operación madura requiere además retry/backoff, locking para evitar ejecuciones solapadas, orden entre jobs dependientes, persistencia del historial y alertas.

17. Observabilidad y métricas operativas

MetricsService

Qué es: Registra duraciones, calidad y resultados del pipeline.

Importancia en este proyecto: Permite saber dónde se consume tiempo y qué etapa aporta o degrada valor.

Valor en otros proyectos: Esencial para optimizar búsqueda y para detectar regresiones de rendimiento.

En ejecuciones del proyecto aparecen etapas como Hybrid Retrieval, LTR Ranking, Reranking, Quality Scoring, Duplicate Detection y Diversity Filtering con tiempos separados. Esa instrumentación permite localizar cuellos de botella, por ejemplo cuando reranking domina la latencia total.

DashboardService

Qué es: Expone o agrega métricas para inspección operativa.

Importancia en este proyecto: Convierte métricas técnicas en una vista usable para validar salud y tendencia.

Valor en otros proyectos: Reutilizable en servicios ML, búsqueda, APIs y sistemas de background jobs.

18. Persistencia de datos y producción

Qdrant / MemoryRepository

Qué es: Persistencia vectorial para memorias y búsqueda por embedding.

Importancia en este proyecto: Da soporte a retrieval semántico a escala y separa almacenamiento de la lógica de ranking.

Valor en otros proyectos: Reutilizable en RAG, recommendation, semantic search y memory services.

KeywordIndex / KeywordIndexLoader

Qué es: Índice léxico en memoria y loader que lo reconstruye desde la persistencia.

Importancia en este proyecto: Permite BM25/keyword retrieval rápido y sincronizado con memorias.

Valor en otros proyectos: Patrón útil cuando se combina un índice derivado en memoria con almacenamiento durable.

Directorio de datos operativo

Qué es: Ubicación unificada para modelos, feedback, conocimiento, grafo, inferencias y estado.

Importancia en este proyecto: Evita pérdida de datos o inconsistencias entre local y contenedor.

Valor en otros proyectos: Reutilizable en cualquier aplicación que mezcle base externa y artefactos locales.

La auditoría detectó una diferencia entre persistencia local y contenedor: modelos/feedback/conocimiento/grafos pueden existir en JSON pero perderse si el directorio no se monta de forma persistente. Esto debe considerarse parte de la implementación, no un detalle de despliegue.

19. API, contratos y seguridad

API Express canonical

Qué es: Camino HTTP activo para memoria, contexto, conocimiento, jobs, métricas y dashboard.

Importancia en este proyecto: Define el contrato externo del servicio.

Valor en otros proyectos: Es la frontera que permite integrar UIs, agentes, CLI u otros servicios.

Validación de payloads

Qué es: Schemas de entrada y errores normalizados.

Importancia en este proyecto: Evita estados inválidos y reduce superficie de ataque.

Valor en otros proyectos: Requisito transversal para cualquier API de producción.

Autenticación / autorización

Qué es: Control de identidad y permisos para endpoints mutables.

Importancia en este proyecto: Protege memoria, feedback, entrenamiento y operaciones de mantenimiento.

Valor en otros proyectos: Crítico en servicios con datos personales o empresariales.

20. Estado global de implementación y estabilización

Área	Estado	Importancia
Retrieval híbrido	Implementado	Núcleo de cobertura multi-fuente
LTR	Implementado	Ranking aprendido e interpretable
Reranking	Implementado	Refinamiento de precisión
Quality / duplicate / diversity	Implementado	Calidad del top-K
Feedback + online learning	Implementado	Adaptación continua
Persistencia LTR	Implementado	Aprendizaje durable
Métricas de ranking	Implementadas en tests	Validación objetiva
Context-aware retrieval	Parcialmente integrado	Personalización/contexto
Knowledge → graph sync	Integración pendiente	Coherencia del grafo
Lifecycle / consolidation	Parcial	Retención y memoria a largo plazo
Cleanup	Pendiente/incompleto	Control de crecimiento
Suite reproducible + CI	Pendiente	Puerta de calidad
Persistencia producción	Requiere estabilización	Supervivencia a restart

21. Importancia de cada implementación en otros proyectos

Implementación	Problema que resuelve	Cuándo reutilizar
Precision/Recall/MRR/NDCG	Medir ranking objetivamente	Cualquier retrieval/recommender/RAG
Hybrid Retrieval	Cobertura semántica + exacta + relacional	Corpus heterogéneos
Weighted RRF	Fusionar rankings heterogéneos	Múltiples motores/índices
LTR	Aprender orden de resultados	Cuando hay feedback o ground truth
FeatureExtractor	Separar señales del modelo	Ranking interpretable/evolutivo
Reranking	Mejorar top-N con coste controlado	Búsqueda de dos etapas
Quality Scoring	Filtrar candidatos débiles	RAG y respuestas confiables
Duplicate Detection	Eliminar redundancia	Memoria acumulativa / ingest repetido
Diversity Filtering	Maximizar cobertura del top-K	Exploración/recomendación/RAG
FeedbackCollector	Capturar señales reales	Productos interactivos
OnlineTrainer	Adaptar rápido	Preferencias cambiantes
Offline Training	Reentrenar reproduciblemente	Release/model governance
ModelRepository	Durabilidad/versionado	Serving ML
ContextModel	Personalización multi-turn	Asistentes/agentes
Knowledge Graph	Relaciones e inferencia	Dominios relacionales
Lifecycle/Consolidation	Memoria de largo plazo sostenible	Agentes persistentes
Scheduler/Jobs	Automatizar mantenimiento	Servicios operativos
Regression Tests	Evitar degradación silenciosa	Cambios frecuentes de ranking

22. Orden recomendado de fases para replicar el sistema

Fase 1 - Retrieval mínimo: Embeddings + vector store + keyword/BM25 + dataset de evaluación.

Fase 2 - Métricas: Precision@K, Recall@K, MRR y NDCG@K antes de optimizar.

Fase 3 - Hybrid + RRF: Combinar fuentes y medir baseline.

Fase 4 - Reranking y calidad: Embedding reranker, quality scoring, duplicates y diversity.

Fase 5 - LTR: FeatureExtractor + modelo + ranker + dataset de feedback.

Fase 6 - Persistencia: ModelRepository, feedback durable, restore tras restart.

Fase 7 - Aprendizaje: TrainingService offline + OnlineTrainer/Optimizer.

Fase 8 - Context-aware: Query profile, strategy, candidate budget y ContextModel.

Fase 9 - Knowledge graph: Extracción, sync, inferencia y graph evidence.

Fase 10 - Lifecycle: Importance, freshness, consolidation, cleanup.

Fase 11 - Operación: Scheduler, jobs, retries, locking, metrics, dashboard.

Fase 12 - Release gate: Regression datasets, CI, persistencia de contenedor, seguridad y contratos.

23. Criterios de aceptación para una versión estable

- El ranking supera o mantiene los umbrales definidos de Precision@K, Recall@K, MRR y NDCG@K frente al baseline.
- Un restart conserva memoria, modelo LTR, feedback, knowledge graph e inferencias.
- Las señales de feedback distinguen impression, click, accept, reject e ignore.
- El pipeline registra latencia por etapa y calidad del resultado final.

- Los tests de regresión son reproducibles y no escriben en datos reales.
- El ContextModel está conectado al camino real sin cambiar el comportamiento cuando no hay contexto.
- Knowledge extraction sincroniza el grafo de forma idempotente.
- Los jobs tienen historial persistente, locking/retry y dependencias ordenadas.
- No existen caminos legacy activos que compitan con el camino canonical.

24. Conclusión

El proyecto ya no debe describirse como una simple aplicación con IA. Es un sistema de retrieval y memoria adaptativa con múltiples fuentes, ranking aprendido, feedback, persistencia, contexto y conocimiento estructurado.

Las implementaciones más reutilizables en otros proyectos son los principios que separan cada responsabilidad: recuperar, fusionar, rankear, rerankear, medir calidad, reducir redundancia, aprender del feedback, persistir el aprendizaje y verificarlo con métricas de regresión.

La ventaja arquitectónica está en que cada fase puede evolucionar sin sustituir todo el sistema: se puede cambiar el embedding, el ranker, el optimizador, el almacenamiento o la estrategia de contexto manteniendo contratos y métricas comparables.

25. Evidencia técnica usada para esta versión

Este documento se ha reconstruido a partir de material técnico del proyecto disponible en los archivos del usuario: auditoría v1, plan de implementación, logs de retrieval/LTR, pruebas de ranking, diffs del container y ejemplos de entrenamiento. Se han priorizado los componentes realmente observados en el proyecto frente a descripciones genéricas.

Entre los hallazgos usados: pipeline Hybrid Retrieval → LTR → Reranking → Quality Scoring → Duplicate Detection → Diversity Filtering; LTR con persistencia y entrenamiento online/offline; métricas Precision@5, Recall@5, MRR y NDCG@5 en pruebas; query profiling y estrategias adaptativas; knowledge graph, context, lifecycle, jobs y riesgos de persistencia/CI.